IU International University of Applied Sciences

Faculty of Computer Science

Module: DLMDSPWP01 – Programming with Python

Name: Pavan Kumar Shashidhara

Matriculation Number: 10242604

Student ID: 5508614

Email: pavan-kumar.shashidhara@iu-study.org

Tutor:-

Rami Alkubaty

Digital Resources:

GitHub Repository: ***https://github.com/Pavannkumar/Pavannkumar-Ideal-Function-Assignment-Python***

Submission Date:- 14/03/2026

Table of Contents

1. Introduction 5

2. Problem Definition 6

3. Data Processing Pipeline Architecture 8

3.1 Data Loading 8

3.2 Database Storage 9

3.3 Ideal Function Selection 9

3.4 Test Data Mapping 9

3.5 Result Storage 10

3.6 Visualization 10

4. Tools and Technologies 10

4.1 Python 10

4.2 Pandas 10

4.3 NumPy 11

4.4 SQLAlchemy 11

4.5 SQLite 11

4.6 Bokeh 11

4.7 Unittest 12

5. Mathematical Methodology 12

5.1 Least Square Method 12

5.2 Ideal Function Selection 13

5.3 Maximum Deviation Calculation 13

5.4 Test Data Mapping Criterion 13

6. System Architecture and Implementation. 13

6.1 Base Function Class 14

6.2 Training Function Class 15

6.3 Ideal Function Class 15

6.4 Test Function Class 15

6.5 Exception Handling 15

6.6 Implementation of Core Classes 16

7. Database Design and Data Management 17

7.1 Database Architecture 17
7.2 Storage of Training and ideal Functions 18
7.3 Storage of Test Data 18
7.4 Storage of Mapping Results 18
8. Core Algorithm Implementation 19
8.1 Ideal Function Selection Using Least Squares 20
8.2 Maximum Deviation Calculation 20
8.3 Mapping Test Observations 21
8.4 Deviation Threshold Criterion 21
8.5 Implementation Workflow 21
9.Visualization and Result Analysis 22
9.1 Visualization of Training and Ideal Functions. 22
9.2 Visualization of Test Data Mapping 22
10 Testing and Validation 23
10.1 Unit Testing 24
10.2 Validation of Algorithm Results 24
10.3 Exception Handling 24
11. Version Control and Development Workflow 25
11.1 Branch-Based Development 25
11.2 Development Stages 25
11.3 Development Workflow Visualization 26
12. Discussion 27
13 Conclusion 28
References 30

List Of Figures

Figure 1: Structure of the training, ideal, and test datasets 7
Figure 2: Data processing pipeline used for function selection and test data mapping 8
Figure 3:Class hierarchy of the object-oriented architecture used in the system 14
Figure 4: Python implementation of the core object-oriented classes 16
Figure 5: Example structure of ideal function data stored in the SQLite database 19
Figure 6:Visualization of training data, selected ideal functions, and mapped test observations 23
Figure 7: Git development workflow showing incremental commits in the develop branch 26

List of Table

Table 1: Database tables used in the analytical system 17

1. Introduction

In modern data-driven environments, large volumes of numerical data are generated through scientific measurements, digital systems, and automated processes. Extracting meaningful patterns from such datasets is a fundamental task in fields such as data science, statistical modelling, and computational analysis. One common challenge in these domains is identifying mathematical functions that best approximate observed data points. Accurately determining such functions enables prediction, analysis, and interpretation of complex relationships between variables.

A widely used statistical approach for identifying optimal mathematical relationships is the least squares method. The least squares method estimates the best-fitting function by minimizing the sum of the squared differences between observed values and predicted values produced by a candidate function. This technique is commonly applied in regression analysis, signal processing, and data modelling due to its computational efficiency and reliability in handling noisy data (Montgomery, Peck and Vining, 2012).

The objective of this assignment is to develop a Python-based analytical system that identifies ideal mathematical functions for a given dataset and maps new observations to those functions. The provided dataset consists of three components: four training datasets, fifty candidate ideal functions, and a test dataset containing independent observations. Each dataset contains x–y pairs representing sampled points of mathematical functions.

The first task of the implemented system is to determine which four ideal functions best represent the training datasets. This is achieved by calculating the sum of squared deviations between each training dataset and all fifty candidate ideal functions. The ideal function that produces the minimum deviation is selected as the best-fitting function for the corresponding training dataset.

After identifying the optimal ideal functions, the system evaluates the test dataset to determine whether each test data point can be assigned to one of the selected functions. The assignment is performed based on a deviation threshold rule derived from the maximum deviation observed during the training phase.

In addition to the mathematical analysis, the implemented solution also demonstrates several software engineering principles, including object-oriented programming, exception

handling, database integration, and automated testing. The program loads datasets, performs least squares analysis, stores results in a SQLite database, and visualizes the mapping results using interactive graphical representations.

The remainder of this report is structured as follows. Section 2 describes the problem definition and dataset structure. Section 3 explains the architecture of the data processing pipeline. Section 4 introduces the tools and technologies used in the implementation. Section 5 describes the mathematical methodology used to select ideal functions and map test data. Section 6 discusses the system architecture and implementation details. Section 7 explains the database design and data management approach. Section 8 presents the visualization and analysis of results. Section 9 discusses testing and validation procedures. Section 10 describes the Git version control workflow used during development. Finally, Section 11 concludes the report and summarizes the key findings.

2. Problem Definition

The objective of this assignment is to design and implement a Python-based analytical system capable of identifying mathematical functions that best approximate a given dataset and mapping new observations to these functions. The assignment focuses on applying statistical analysis techniques, database integration, and data visualization to analyze relationships between datasets.

The system must analyze three datasets provided as comma-separated value (CSV) files. These datasets include a training dataset, an ideal functions dataset, and a test dataset. Each dataset contains numerical values representing sampled points of mathematical functions.

The training dataset contains four dependent variables, labelled, y_1, y_2, y_3, and y_4, together with a shared independent variable x. These training functions represent observed data patterns that must be approximated by selecting suitable functions from a larger set of candidate functions.

The ideal functions dataset contains fifty possible functions that may approximate the training datasets. Each ideal function is represented as a series of x-y value pairs. The task of the program is to determine which four of these ideal functions best match the training datasets.

The selection of the optimal ideal functions is performed using the least squares method. For each training function, the program computes the sum of squared deviations between the training values and the corresponding values of each candidate ideal function. The ideal function that produces the smallest deviation is considered the best-fitting function.

Once the optimal functions have been identified, the system evaluates the test dataset. The test dataset contains independent observations consisting of x and y values that may correspond to one of the selected ideal functions.

To determine whether a test observation can be assigned to a selected ideal function, the program calculates the deviation between the observed y value and the predicted value produced by the ideal function. A test data point is assigned to the ideal function if the deviation does not exceed the maximum deviation observed during the training phase multiplied by a tolerance factor $\sqrt{2}$.

This mapping rule ensures that only test observations within an acceptable deviation range are assigned to the selected ideal functions. Observations that exceed the deviation threshold are not assigned to any function.
Figure 1 illustrates the structure of the datasets used in the assignment, including the training dataset containing four functions, the ideal functions dataset containing fifty candidate functions, and the test dataset containing independent observations.

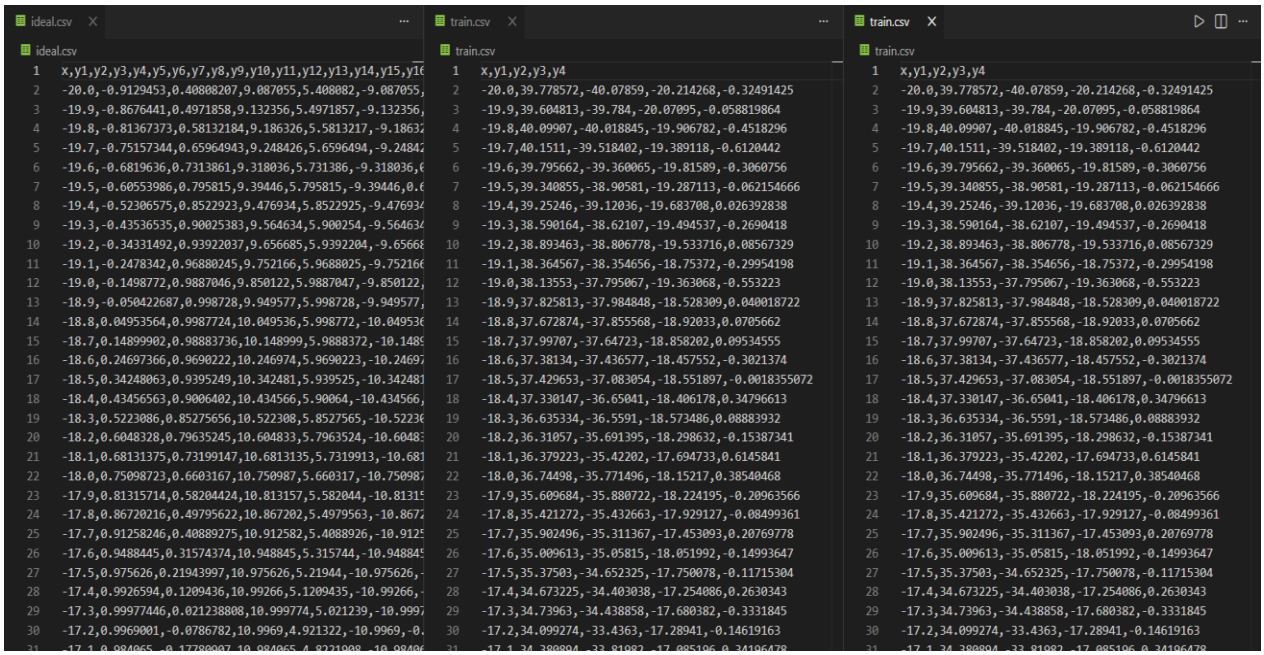

Figure 1: Structure of the training, ideal, and test datasets

3. Data Processing Pipeline Architecture

To ensure a clear and structured workflow, the implemented system follows a data processing pipeline architecture. A pipeline architecture divides the system into a sequence of stages, where each stage performs a specific operation on the data before passing the results to the next stage. This approach improves modularity, maintainability, and transparency of the data analysis process.

The analytical system developed for this assignment consists of six major stages: data loading, database storage, ideal function selection, test data mapping, result storage, and visualization. Each stage contributes to the overall objective of identifying ideal functions and mapping test observations.

The workflow of the implemented system is illustrated in Figure 2, which presents the sequence of processing stages applied to the datasets.

Figure 2: Data processing pipeline used for function selection and test data mapping

3.1 Data Loading

The first stage of the pipeline involves loading the datasets into the Python environment. The datasets are provided as comma-separated value (CSV) files containing numerical

observations of mathematical functions. The program loads these datasets using the Pandas library, which provides efficient data structures for handling tabular data (McKinney, 2022)

Three datasets are loaded during this stage: the training dataset containing four functions, the ideal functions dataset containing fifty candidate functions, and the test dataset containing independent observations. The program also validates the datasets to ensure that they are not empty and contain the required columns.

3.2 Database Storage

After loading the datasets, the system stores them in a SQLite database. SQLite is a lightweight relational database system that is well suited for local data storage and small-scale applications (Owens, Allen and Kennedy, 2010).

Database interactions are handled using the SQLAlchemy library, which provides an object-relational mapping interface for Python applications. The training dataset and ideal functions dataset are inserted into separate database tables, allowing efficient storage and retrieval during later stages of the pipeline.

3.3 Ideal Function Selection

In the third stage of the pipeline, the system determines which ideal functions best approximate the training datasets. This is achieved using the least squares method, which minimizes the sum of squared deviations between two functions.

For each training function, the program compares it with all fifty candidate ideal functions and computes the total squared deviation between their corresponding values. The ideal function with the smallest deviation is selected as the best match for the training dataset.

3.4 Test Data Mapping

Once the optimal ideal functions have been identified, the program evaluates the test dataset. Each test observation is compared with the predicted value of the selected ideal functions.

The mapping process determines whether a test observation can be assigned to one of the selected functions based on a deviation threshold rule derived from the training phase.

3.5 Result Storage

After completing the mapping process, the results are stored in the database. Each mapped observation includes the x-value, observed y-value, deviation from the ideal function, and the index of the assigned ideal function.

Storing the results in the database ensures that the mapping outcomes can be analyzed, verified, and reused for further evaluation.

3.6 Visualization

The final stage of the pipeline involves visualizing the relationships between the training datasets, the selected ideal functions, and the mapped test observations. Visualization helps interpret the results and verify whether the selected ideal functions accurately represent the training data.

The visualization is implemented using the Bokeh library, which provides interactive graphical representations that allow users to explore the data and observe the mapping results.

4. Tools and Technologies

The implementation of the analytical system relies on several Python libraries and software tools that support data processing, numerical computation, database management, and visualization. These tools enable the development of a robust pipeline capable of processing datasets efficiently and producing meaningful analytical results.

4.1 Python

Python was selected as the primary programming language for this assignment due to its flexibility, readability, and extensive ecosystem of scientific computing libraries. Python is widely used in data science and statistical computing because it provides powerful libraries that simplify complex numerical and analytical tasks.

Its object-oriented capabilities also make it suitable for implementing modular and maintainable software architectures (Van Rossum and Drake, 2009).

4.2 Pandas

The Pandas library is used for loading and manipulating structured datasets. Pandas provides the DataFrame data structure, which allows efficient storage, filtering, and

transformation of tabular data. In this assignment, Pandas is used to load CSV files containing the training dataset, ideal functions dataset, and test dataset.

The library also simplifies data manipulation tasks required during the least squares computation and test data mapping processes (McKinney, 2022).

4.3 NumPy

The **NumPy library** provides optimized numerical operations for scientific computing. NumPy enables vectorized mathematical computations that operate on entire arrays rather than individual elements, significantly improving performance.

In the implemented system, NumPy is used to compute squared deviations and perform mathematical calculations required for the least squares method (Harris et al., 2020).

4.4 SQLAlchemy

The assignment requires that datasets and mapping results be stored in a relational database. For this purpose, the SQLAlchemy library is used to interact with a SQLite database.

SQLAlchemy provides an object-relational mapping interface that allows Python programs to communicate with relational databases using Python objects instead of raw SQL queries. This simplifies database creation, table management, and data insertion operations (Bayer, 2012).

4.5 SQLite

SQLite is used as the underlying database system for storing the datasets and mapping results. SQLite is a lightweight relational database that operates without a dedicated server, making it suitable for local applications and small-scale analytical systems.

Its simplicity and efficiency make it an ideal choice for storing structured data generated during the analysis process (Owens, Allen and Kennedy, 2010).

4.6 Bokeh

Data visualization is implemented using the Bokeh library, which enables the creation of interactive graphical visualizations. Bokeh generates HTML-based plots that allow users to explore the data using interactive features such as zooming and panning.

In this project, Bokeh is used to visualize the relationships between training functions,

selected ideal functions, and mapped test observations. These visualizations help validate the correctness of the function selection and mapping process (Bokeh Development Team, 2024).

4.7 Unittest

To ensure correctness and reliability of the implemented algorithms, the Python unittest framework is used to perform automated testing.

Unit testing verifies that key components of the system, such as deviation calculations and least squares computations, behave as expected. This helps identify potential implementation errors and ensures that the system produces accurate results (Myers, Sandler and Badgett, 2011).

5. Mathematical Methodology

The core analytical component of the implemented system is based on the least squares method, which is widely used in statistical modelling to determine the best-fitting function for a given dataset. The least squares method minimizes the total squared deviation between observed data points and the values predicted by a candidate function.

In this assignment, the least squares approach is applied to determine which of the fifty candidate ideal functions best approximates each of the four training datasets.

5.1 Least Square Method

The least squares method identifies the function that minimizes the sum of squared deviations between two datasets. For each training dataset and ideal function pair, the deviation is calculated using the following formula:

$$SSD = \sum_{i=1}^{n} (y_{train,i} - y_{ideal,i})^2$$

where:

- $y_{train,i}$ represents the value of the training function at observation i
- $y_{ideal,i}$ represents the corresponding value of the ideal function
- n represents the number of observations

The ideal function that produces the smallest value of SSD is considered the best approximation of the training dataset (Montgomery, Peck and Vining, 2012).

5.2 Ideal Function Selection

The implemented algorithm compares each training dataset with all fifty candidate ideal functions. For every pair of functions, the sum of squared deviations is computed.

The ideal function that minimizes this deviation is selected as the optimal function:

$$f^* = \arg\min_f \sum (y_{train} - y_{ideal})^2$$

This procedure is repeated independently for each of the four training functions.

5.3 Maximum Deviation Calculation

After selecting the best-fitting ideal function for each training dataset, the algorithm calculates the maximum absolute deviation between the training values and the corresponding ideal function values.

$$\delta_{max} = \max |y_{train} - y_{ideal}|$$

This value represents the largest deviation observed during the training phase and is used to determine the tolerance threshold when evaluating test data points.

5.4 Test Data Mapping Criterion

Once the optimal ideal functions are determined, the algorithm evaluates the test dataset. For each test observation (x, y),the predicted value of the selected ideal function is computed and compared with the observed value.

A test data point is assigned to an ideal function only if the deviation satisfies the following condition:

$$|y_{test} - f(x)| \leq \delta_{max} \times \sqrt{2}$$

This threshold ensures that only observations within an acceptable deviation range are mapped to the corresponding ideal function. Observations exceeding this threshold are excluded from the mapping process.

6. System Architecture and Implementation.

The analytical system developed for this assignment follows a modular object-oriented architecture. Object-oriented programming (OOP) is widely used in software engineering

because it improves code organization, modularity, and maintainability by structuring programs into reusable components known as classes (Gamma et al., 1994).

The implemented system divides the functionality of the application into several classes that represent different components of the analytical workflow. Each class is responsible for handling a specific dataset or computational task, allowing the system to maintain a clear separation of responsibilities.
The overall class hierarchy used in the implementation is illustrated in Figure 3.

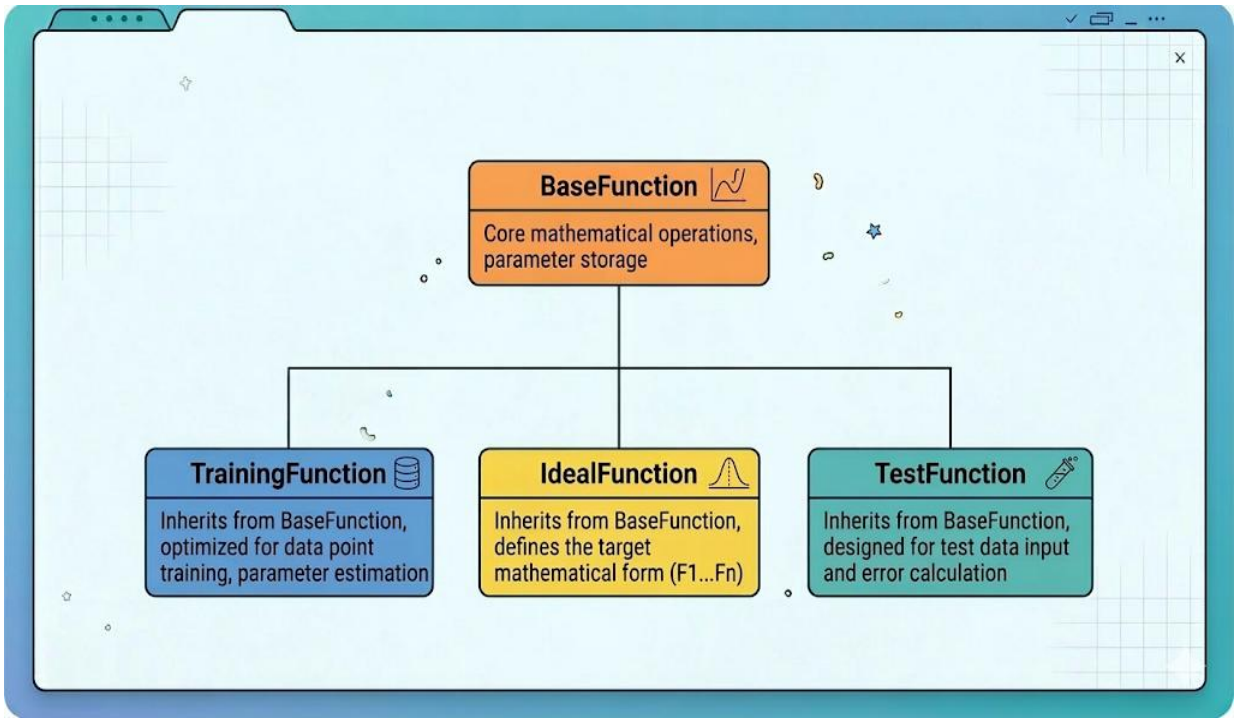

Figure 3:Class hierarchy of the object-oriented architecture used in the system

6.1 Base Function Class

The BaseFunction class serves as the parent class for all function-related components in the system. It defines the fundamental properties and behaviors shared by different types of functions used in the analysis.

This class is responsible for loading datasets, storing x–y values, and providing basic operations that can be reused by derived classes. By centralizing these shared functionalities, the BaseFunction class reduces code duplication and simplifies maintenance.

Inheritance allows the derived classes to extend the base functionality while implementing specialized behaviors for specific datasets.

6.2 Training Function Class

The TrainingFunction class represents the four training datasets provided in the assignment. These datasets contain observed values that must be approximated using candidate ideal functions.

This class inherits common functionality from the BaseFunction class while introducing additional methods for computing deviations between the training dataset and candidate ideal functions.

The class also manages the process of identifying the best-fitting ideal function using the least squares method.

6.3 Ideal Function Class

The IdealFunction class represents the set of fifty candidate functions provided in the ideal dataset. Each ideal function is stored as a series of x–y values that can be compared with the training datasets.

This class provides functionality for retrieving ideal function values, performing comparisons with training datasets, and computing deviations used in the function selection process.
By encapsulating the ideal function logic within a dedicated class, the system ensures that the selection algorithm remains modular and easy to extend.

6.4 Test Function Class

The TestFunction class handles the processing of the test dataset. The test dataset contains independent observations that must be mapped to the selected ideal functions if they satisfy the deviation threshold rule.

This class evaluates each test observation by comparing the observed value with the predicted value of the selected ideal functions. If the deviation falls within the acceptable threshold, the observation is assigned to the corresponding ideal function.

6.5 Exception Handling

To ensure robustness and reliability, the system implements structured exception handling mechanisms. Exception handling is used to detect and manage potential errors such as missing files, invalid dataset structures, or unexpected runtime conditions.

Custom exception classes are implemented to provide clear error messages and prevent

program crashes. This improves the stability of the system and ensures that errors are handled in a controlled and informative manner.

6.6 Implementation of Core Classes

The object-oriented architecture described previously is implemented through several Python classes that encapsulate the functionality required for dataset handling and analytical computation. Each class extends the BaseFunction class and introduces specialized behavior for training data processing, ideal function comparison, and test data mapping.

Figure 4 illustrates a simplified excerpt of the Python implementation showing the inheritance structure and key methods used to compute deviations, perform least squares calculations, and evaluate test observations.

```python
# Base function class
class BaseFunction:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def deviation(self, other_y):
        return abs(self.y - other_y)

#  a training function used for least-squares fitting
class TrainingFunction(BaseFunction):
    def max_deviation(self, ideal_y):
        return abs(self.y - ideal_y).max()


#  an ideal reference function
class IdealFunction(BaseFunction):
    def squared_error(self, training_y):
        return ((training_y - self.y) ** 2).sum()


#  a test data point.
class TestFunction(BaseFunction):
    def is_assignable(self, ideal_y, max_dev):
        return abs(self.y - ideal_y) <= max_dev
```

Figure 4: Python implementation of the core object-oriented classes

The BaseFunction class provides shared functionality for storing function values and calculating deviations between datasets. The TrainingFunction class extends this functionality to compute the maximum deviation used during the least squares fitting process. The IdealFunction class performs squared error calculations used to identify the best-fitting ideal function, while the TestFunction class determines whether a test observation satisfies the deviation threshold required for mapping to an ideal function.

7. Database Design and Data Management

Efficient data storage and retrieval are essential components of the analytical system developed in this assignment. To ensure structured storage and reliable access to datasets and analysis results, the system uses a relational database implemented with SQLite. SQLite is a lightweight, serverless database engine that is widely used for local data storage in data-driven applications (Owens, Allen and Kennedy, 2010).

The interaction between the Python program and the database is handled using the SQLAlchemy library, which provides an object-relational mapping (ORM) interface. This allows database tables and records to be manipulated directly through Python objects rather than writing raw SQL queries. The use of SQLAlchemy simplifies database management and ensures seamless integration between the analytical program and the underlying database system.

The database is used to store the training datasets, the candidate ideal functions, the test observations, and the results produced during the mapping process. Organizing these datasets within a relational database ensures consistency, facilitates data retrieval, and supports the analytical workflow implemented in the system.

7.1 Database Architecture

The database structure is designed to store the datasets used throughout the analytical process in separate tables. Each table represents a specific dataset involved in the function selection and test data mapping workflow. The database schema ensures that the data required for training, comparison, and evaluation can be accessed efficiently during different stages of the computation.

Table 1 summarizes the database tables used in the system, including their purpose and the data stored within each table.

Table 1: Database tables used in the analytical system

Table Name	Description	Stored Data
training_data	**Stores the four training functions used for ideal function selection.**	**x, y1 ,y2 , y3, y4**
ideal_functions	**Contains fifty candidate ideal functions used for least-squares comparison.**	**x, y1–y50**

test_data	**Stores the independent test observations used for evaluation.**	**x, y**
mapped_results	**Stores mapping results between test points and ideal functions.**	**x, y, deviation, ideal_function**

7.2 Storage of Training and ideal Functions

The training dataset consists of four functions that represent observed data patterns. These functions are stored in the table *training_data*, where each record contains an x-value and the corresponding values for the four training functions y1,y2,y3,y4. These functions are later compared with the candidate ideal functions using the least squares method in order to identify the best-fitting functions.

The ideal functions dataset contains fifty candidate functions that serve as potential approximations for the training datasets. These functions are stored in the table *ideal_functions*, where each record contains the x-value and the corresponding values for the candidate functions y1 to y50. Storing these values in the database allows the analytical system to efficiently retrieve and compare them during the function selection process.

7.3 Storage of Test Data

The test dataset contains independent observations that must be evaluated against the selected ideal functions. These observations are stored in the table *test_data*, where each record contains an x-value and the corresponding observed y-value.

During the evaluation phase, the system retrieves these observations and compares them with the predicted values of the selected ideal functions. This comparison is used to determine whether the test data points satisfy the deviation threshold required for mapping.

7.4 Storage of Mapping Results

After evaluating the test observations, the system stores the mapping results in the table mapped_results. This table records the relationship between each test data point and the ideal function to which it is assigned.

Each record in the mapping results table includes the x-value of the test observation, the observed y-value, the deviation between the observed value and the ideal function value, and the identifier of the corresponding ideal function. Storing the mapping results allows the

system to track which test points satisfy the assignment criteria and provides a basis for further analysis and visualization.

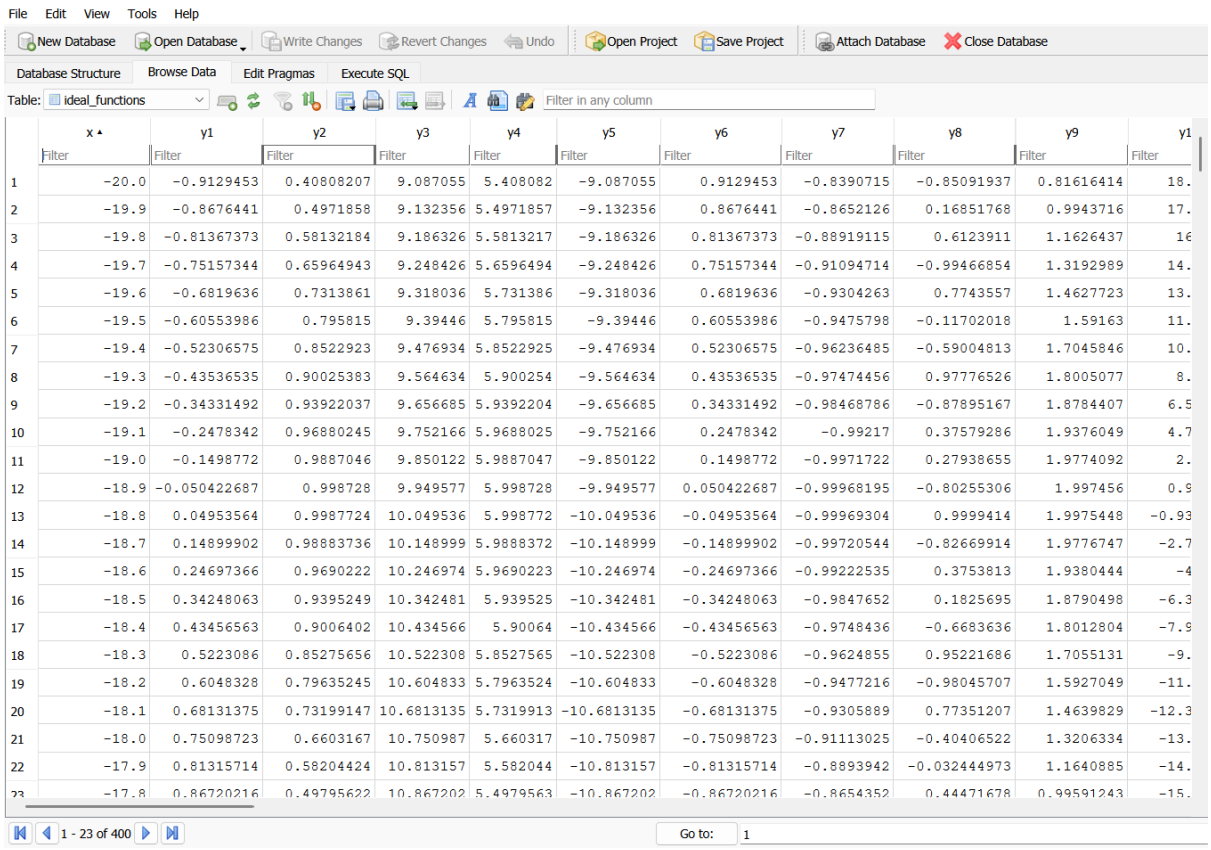

Table: ideal_functions

	x	y1	y2	y3	y4	y5	y6	y7	y8	y9	y1
1	-20.0	-0.9129453	0.40808207	9.087055	5.408082	-9.087055	0.9129453	-0.8390715	-0.85091937	0.81616414	18.
2	-19.9	-0.8676441	0.4971858	9.132356	5.4971857	-9.132356	0.8676441	-0.8652126	0.16851768	0.9943716	17.
3	-19.8	-0.81367373	0.58132184	9.186326	5.5813217	-9.186326	0.81367373	-0.88919115	0.6123911	1.1626437	16
4	-19.7	-0.75157344	0.65964943	9.248426	5.6596494	-9.248426	0.75157344	-0.91094714	-0.99466854	1.3192989	14.
5	-19.6	-0.6819636	0.7313861	9.318036	5.731386	-9.318036	0.6819636	-0.9304263	0.7743557	1.4627723	13.
6	-19.5	-0.60553986	0.795815	9.39446	5.795815	-9.39446	0.60553986	-0.9475798	-0.11702018	1.59163	11.
7	-19.4	-0.52306575	0.8522923	9.476934	5.8522925	-9.476934	0.52306575	-0.96236485	-0.59004813	1.7045846	10.
8	-19.3	-0.43536535	0.90025383	9.564634	5.900254	-9.564634	0.43536535	-0.97474456	0.97776526	1.8005077	8.
9	-19.2	-0.34331492	0.93922037	9.656685	5.9392204	-9.656685	0.34331492	-0.98468786	-0.87895167	1.8784407	6.5
10	-19.1	-0.2478342	0.96880245	9.752166	5.9688025	-9.752166	0.2478342	-0.99217	0.37579286	1.9376049	4.7
11	-19.0	-0.1498772	0.9887046	9.850122	5.9887047	-9.850122	0.1498772	-0.9971722	0.27938655	1.9774092	2.
12	-18.9	-0.050422687	0.998728	9.949577	5.998728	-9.949577	0.050422687	-0.99968195	-0.80255306	1.997456	0.9
13	-18.8	0.04953564	0.9987724	10.049536	5.998772	-10.049536	-0.04953564	-0.99969304	0.9999414	1.9975448	-0.93
14	-18.7	0.14899902	0.98883736	10.148999	5.9888372	-10.148999	-0.14899902	-0.99720544	-0.82669914	1.9776747	-2.7
15	-18.6	0.24697366	0.9690222	10.246974	5.9690223	-10.246974	-0.24697366	-0.99222535	0.3753813	1.9380444	-4
16	-18.5	0.34248063	0.9395249	10.342481	5.939525	-10.342481	-0.34248063	-0.9847652	0.1825695	1.8790498	-6.3
17	-18.4	0.43456563	0.9006402	10.434566	5.90064	-10.434566	-0.43456563	-0.9748436	-0.6683636	1.8012804	-7.9
18	-18.3	0.5223086	0.85275656	10.522308	5.8527565	-10.522308	-0.5223086	-0.9624855	0.95221686	1.7055131	-9.
19	-18.2	0.6048328	0.79635245	10.604833	5.7963524	-10.604833	-0.6048328	-0.9477216	-0.98045707	1.5927049	-11.
20	-18.1	0.68131375	0.73199147	10.6813135	5.7319913	-10.6813135	-0.68131375	-0.9305889	0.77351207	1.4639829	-12.3
21	-18.0	0.75098723	0.6603167	10.750987	5.660317	-10.750987	-0.75098723	-0.91113025	-0.40406522	1.3206334	-13.
22	-17.9	0.81315714	0.58204424	10.813157	5.582044	-10.813157	-0.81315714	-0.8893942	-0.032444973	1.1640885	-14.
23	-17.8	0.86720216	0.49795622	10.867202	5.4979563	-10.867202	-0.86720216	-0.8654352	0.44471678	0.99591243	-15.

1 - 23 of 400 Go to: 1

Figure 5: Example structure of ideal function data stored in the SQLite database

The database structure shown in Figure 5 illustrates how the ideal function dataset is stored in the SQLite database. Each row represents a specific x-value, while the columns represent the corresponding values of the candidate ideal functions. This structured representation enables efficient retrieval of function values during the least squares comparison and the test data mapping processes.

8. Core Algorithm Implementation

The core functionality of the analytical system is based on an algorithm that identifies the most suitable ideal functions for the given training datasets and maps the test observations to these functions based on a defined deviation threshold. The implementation of this algorithm follows a structured workflow consisting of three main stages: ideal function selection using the least squares method, calculation of maximum deviation, and mapping of test observations to the selected ideal functions.

The algorithm operates on the datasets stored in the SQLite database and uses Python's data processing libraries to perform numerical computations and comparisons efficiently.

8.1 Ideal Function Selection Using Least Squares

The first step of the algorithm involves identifying the ideal functions that best approximate the four training datasets. Since the ideal dataset contains fifty candidate functions, each training function must be compared with all candidate functions to determine the closest match.

This comparison is performed using the least squares method, which measures the similarity between two functions by calculating the sum of squared differences between their corresponding values.
The mathematical formulation of the least squares error is given by

$$E = \sum_{i=1}^{n} (y_{train,i} - y_{ideal,i})^2$$

where:

- $y_{train,i}$ represents the value of the training function at position i
- $y_{ideal,i}$ represents the corresponding value of the candidate ideal function
- n represents the number of data points

For each training function, the algorithm computes this error for all fifty ideal functions. The ideal function that produces the smallest error value is selected as the best-fitting function. This process is repeated independently for the four training functions$y_{1,}y_2y_{3,}y_{4,}$.

8.2 Maximum Deviation Calculation

After determining the best-fitting ideal function for each training dataset, the algorithm calculates the maximum deviation between the training values and the corresponding ideal function values.
The maximum deviation is defined as:

$$\delta_{max} = \max |y_{train} - y_{ideal}|$$

This value represents the largest difference observed between the training data and the selected ideal function across all data points.

The calculated maximum deviation is used to determine the tolerance threshold when mapping test observations to the ideal functions.

8.3 Mapping Test Observations

Once the best-fitting ideal functions have been identified, the algorithm evaluates the test dataset to determine whether the test observations can be assigned to any of the selected ideal functions.

For each test observation (x , y) (x,y), the algorithm retrieves the corresponding value of the selected ideal function at the same x-coordinate. The difference between the observed value and the predicted ideal value is then calculated.
The deviation is computed as:

$$\Delta y = |y_{test} - y_{ideal}|$$

This value represents how far the test observation deviates from the predicted ideal function.

8.4 Deviation Threshold Criterion

A test observation is assigned to an ideal function only if the deviation satisfies the assignment condition specified in the problem statement. The threshold is determined by multiplying the maximum deviation obtained during the training phase by the factor $\sqrt{2}$.
The assignment condition is therefore defined as:

$$|y_{test} - f(x)| \leq \delta_{max} \times \sqrt{2}$$

If this condition is satisfied, the test observation is mapped to the corresponding ideal function. Otherwise, the observation is not assigned to any function.

This criterion ensures that only observations that fall within an acceptable deviation range are considered valid mappings.

8.5 Implementation Workflow

The algorithm implemented in the Python program follows the workflow below:

1. Load training, ideal, and test datasets from CSV files.
2. Store the datasets in the SQLite database.
3. Compare each training function with all fifty ideal functions using least squares.
4. Select the ideal function with the minimum error for each training dataset.
5. Calculate the maximum deviation between each training function and its corresponding ideal function.
6. Evaluate each test observation and compute its deviation from the selected ideal functions.

7. Assign the observation to an ideal function if the deviation satisfies the threshold condition.
8. Store the mapping results in the database for further analysis and visualization.

This structured approach ensures that the system systematically identifies the best-fitting functions and accurately maps test observations based on the defined deviation criteria.

9.Visualization and Result Analysis

Data visualization plays an important role in understanding the relationships between the training datasets, the selected ideal functions, and the mapped test observations. Visualization enables the analytical results to be interpreted more clearly by illustrating how closely the selected ideal functions approximate the training datasets and how the test data points align with these functions.

In this project, visualization is implemented using the Bokeh library, which provides interactive plotting capabilities in Python. Bokeh allows the creation of dynamic visual representations of datasets and supports features such as zooming, panning, and tooltips that help users explore the plotted data more effectively (Bokeh Development Team, 2024).

9.1 Visualization of Training and Ideal Functions.

The visualization combines the four training functions with their corresponding selected ideal functions. Each training function is plotted using a distinct color to differentiate it from the other datasets. The ideal functions selected through the least squares algorithm are plotted alongside the training functions to illustrate the degree of similarity between the observed and predicted values.

This graphical representation helps validate whether the selected ideal functions closely follow the behaviour of the training datasets. When the least squares selection process works correctly, the ideal functions should closely overlap with the corresponding training data points.

9.2 Visualization of Test Data Mapping

In addition to the training and ideal functions, the visualization also includes the mapped test observations. These test points are plotted on the same graph and are represented using distinct markers to distinguish them from the training datasets.

The mapping of test observations is based on the deviation threshold criterion described earlier. When a test point satisfies the mapping condition, it is assigned to the corresponding

ideal function and displayed on the visualization. This representation allows users to observe how the test points relate to the selected ideal functions.

By plotting the test observations together with the training and ideal functions, it becomes easier to verify whether the algorithm correctly identifies valid mappings and excludes observations that fall outside the permitted deviation range.

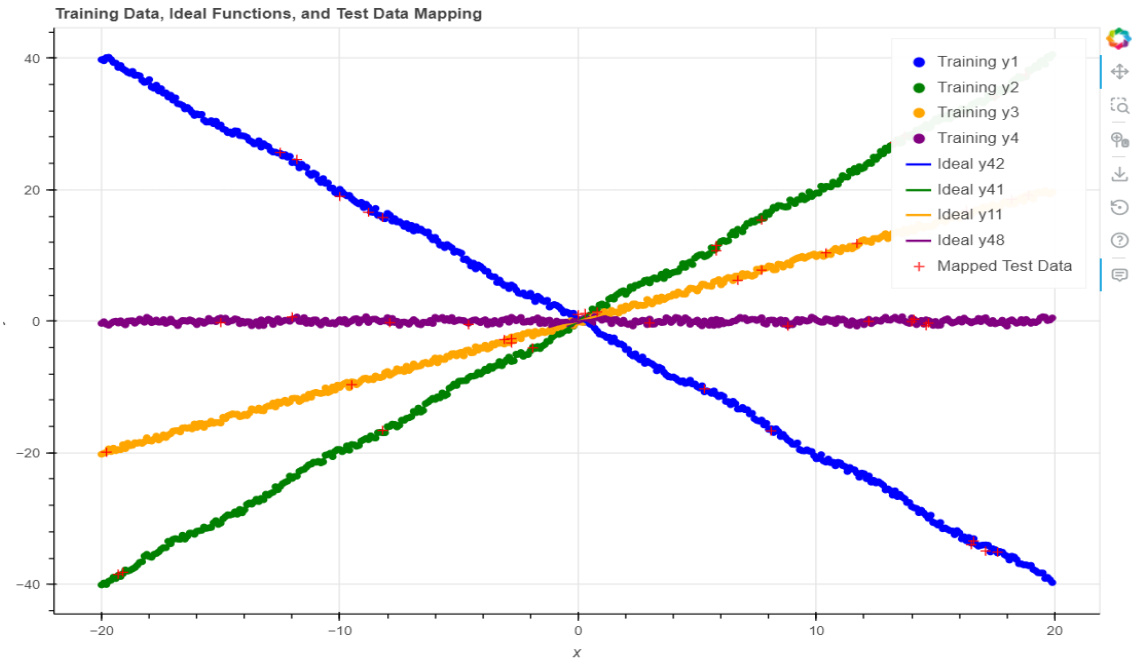

Figure 6:Visualization of training data, selected ideal functions, and mapped test observations

The visualization shown in Figure 6 illustrates the relationship between the training datasets, the selected ideal functions, and the mapped test observations. The training functions form the reference patterns, while the ideal functions represent the best-fitting mathematical approximations identified by the least squares algorithm. The mapped test observations appear as individual points that align with the corresponding ideal functions when the deviation threshold condition is satisfied.

This graphical representation provides a clear confirmation that the analytical system correctly identifies the optimal ideal functions and accurately maps the test data points according to the defined assignment criteria.

10 Testing and Validation

Testing and validation are essential steps in the development of reliable software systems. In this project, testing was performed to ensure that the implemented algorithms operate

correctly and produce accurate results when processing the training, ideal, and test datasets. The testing process focuses on verifying the correctness of the deviation calculations, the least squares function selection procedure, and the mapping of test observations to the selected ideal functions.

To support systematic testing, the Python unittest framework was used to implement automated unit tests for key components of the system. Unit testing is a widely used software testing technique that allows individual components of a program to be verified independently (Myers, Sandler and Badgett, 2011).

10.1 Unit Testing

The system includes a dedicated test module that verifies the functionality of the implemented classes and methods. These tests ensure that the mathematical computations and mapping logic behave as expected.
Unit tests were implemented to validate several important aspects of the system, including:

- correct calculation of deviations between datasets
- accurate computation of squared errors used in the least squares method
- correct identification of assignable test observations
- verification of threshold conditions used for mapping test data

By executing these tests, the system confirms that the implemented algorithms operate correctly under normal conditions.

10.2 Validation of Algorithm Results

In addition to automated unit tests, the correctness of the algorithm was validated by analyzing the mapping results and visualizing the relationships between the datasets. The selected ideal functions were verified to closely approximate the training datasets, confirming that the least squares selection process operates correctly.

The visualization of mapped test observations further confirms the correctness of the algorithm. Test points that satisfy the deviation threshold appear aligned with the selected ideal functions, while points that exceed the threshold are not assigned. This behavior demonstrates that the mapping rule is applied consistently across all observations.

10.3 Exception Handling

To improve the robustness of the system, custom exception handling mechanisms were implemented to detect and manage potential runtime errors. These exceptions help ensure

that invalid data inputs or unexpected conditions do not cause the program to fail unexpectedly.

Examples of handled conditions include:

- missing or empty dataset files
- incorrect data formats
- errors during database insertion or retrieval

When such conditions occur, the program raises descriptive error messages that allow the user to identify and resolve the issue. This approach improves the reliability and maintainability of the analytical system.

11. Version Control and Development Workflow

The development of the analytical system was managed using Git, a distributed version control system widely used in software development to track code changes and support collaborative development (Chacon and Straub, 2014). Version control allows developers to maintain a history of modifications, revert to previous versions if necessary, and organize development work into structured branches.

In this project, Git was used to maintain the project repository and track the incremental development of the system. Each significant stage of the implementation process was recorded through commits, allowing the evolution of the program to be clearly documented.

11.1 Branch-Based Development

To ensure organized development, a branch-based workflow was adopted. In this approach, the main development work was performed in a separate branch known as the develop branch, while the main branch was reserved for stable versions of the project.

The develop branch allowed new features and improvements to be implemented incrementally without affecting the stable version of the program. This workflow provides a controlled environment for testing and refining the implementation before integrating changes into the final version of the system.

11.2 Development Stages

During the development process, the implementation progressed through several stages. Each stage was committed to the repository to maintain a clear record of progress. These stages included:

- placing and organizing the required datasets
- implementing the custom exception handling classes

- developing the core Python classes for function processing
- creating the database integration and data storage functionality
- implementing the visualization component
- adding automated unit tests
- finalizing the execution of the complete analytical pipeline

By maintaining these incremental commits, the development history reflects the structured progression of the system from initial setup to final implementation.

11.3 Development Workflow Visualization

The Git commit history provides a visual overview of the development process and the sequence of implemented features. This history illustrates how the project evolved through multiple development stages before reaching the final working version.

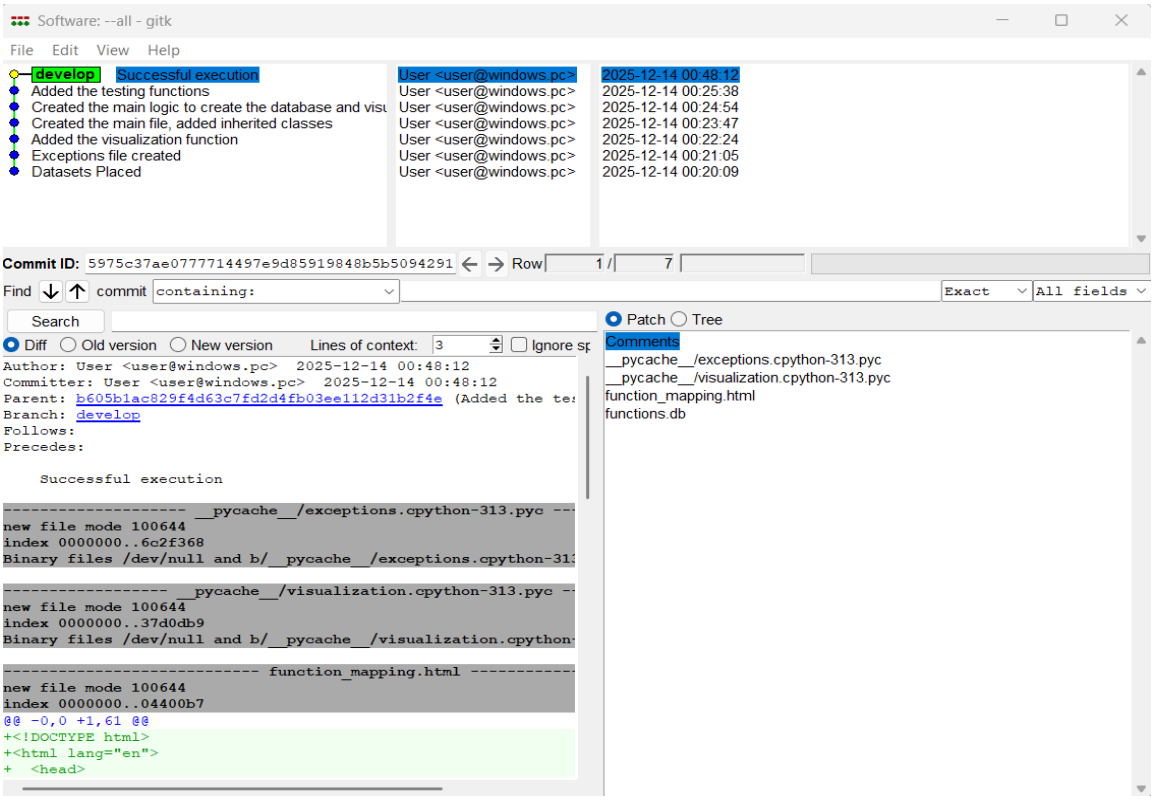

Figure 7: Git development workflow showing incremental commits in the develop branch

The commit history shown in Figure 7 illustrates the development workflow followed during the implementation of the analytical system. Each commit corresponds to a specific stage of development, such as dataset preparation, class implementation, visualization development, and testing integration. This structured commit history demonstrates the iterative development process and highlights the use of version control to track the evolution of the project.

12. Discussion

The implementation of the analytical system successfully demonstrates how mathematical modeling and data processing techniques can be used to identify relationships between datasets. By applying the least squares method, the system effectively determines which ideal functions best approximate the given training datasets and maps test observations according to the defined deviation threshold.

The results obtained from the algorithm indicate that the selected ideal functions closely follow the behavior of the training datasets. This confirms that the least squares approach is an appropriate method for identifying optimal function approximations within the provided candidate set. The visualization of the results further supports this observation, as the selected ideal functions closely overlap with the corresponding training functions across the range of x-values.

The mapping of test observations also demonstrates the effectiveness of the implemented deviation threshold criterion. Test points that fall within the allowed deviation range are successfully assigned to the corresponding ideal functions, while points that exceed the threshold are excluded. This ensures that only observations that reasonably align with the predicted functions are considered valid mappings.

Another strength of the implemented system is the modular architecture achieved through object-oriented programming. The use of separate classes for training functions, ideal functions, and test observations improves the clarity and maintainability of the code. This design allows different components of the analytical process to be managed independently while still interacting effectively within the overall system.

The integration of a relational database using SQLite also contributes to the efficiency of the implementation. Storing datasets and mapping results in the database allows structured data management and facilitates data retrieval during different stages of the computation. The use of SQLAlchemy further simplifies the interaction between Python and the database, improving the overall flexibility of the system.

However, several limitations should also be acknowledged. The algorithm assumes that the candidate ideal functions provided in the dataset contain suitable approximations for the training functions. If the ideal dataset does not include appropriate functions, the least

squares method would still select the closest available match, which may not accurately represent the underlying data pattern.

Another limitation is that the system relies on discrete data points rather than continuous function representations. This means that comparisons are performed only at the available x-values, which may limit the precision of the approximation in cases where the dataset resolution is low.

Despite these limitations, the implemented system demonstrates a practical approach to solving the assignment problem by combining mathematical analysis, database management, and visualization techniques within a structured software architecture.

13 Conclusion

This project presented the development of a Python-based analytical system designed to identify ideal mathematical functions that best approximate a set of training datasets and to map independent test observations to these functions. The implementation combined mathematical modeling, database management, and data visualization techniques within a structured software architecture.

The least squares method was applied to determine which of the fifty candidate ideal functions most closely approximated each of the four training datasets. By calculating the sum of squared deviations between the training data and the ideal functions, the algorithm was able to identify the most suitable function for each dataset. This approach provided a systematic method for selecting optimal function approximations.

Following the function selection stage, the system evaluated the test dataset and applied a deviation threshold criterion to determine whether individual observations could be mapped to the selected ideal functions. Only observations that satisfied the defined deviation condition were assigned, ensuring that the mapping process remained consistent with the problem requirements.

The implementation also demonstrated the importance of structured software design. The use of object-oriented programming enabled the system to organize different components of the analytical process into reusable classes. In addition, the integration of an SQLite database allowed the datasets and mapping results to be stored in a structured format that supports efficient retrieval and analysis.

Visualization of the training datasets, selected ideal functions, and mapped test observations provided a clear representation of the analytical results. These visualizations confirmed that the selected ideal functions closely approximate the training data and that the mapping algorithm correctly assigns test observations within the allowed deviation range.

Overall, the project demonstrates how data analysis techniques, mathematical modeling, and software engineering principles can be combined to develop a functional analytical system. The implemented solution successfully fulfills the requirements of the assignment while providing a clear framework for performing function approximation and dataset mapping tasks.

References

Bayer, M., 2012. *SQLAlchemy documentation*. Available at: https://www.sqlalchemy.org (Accessed: 18 March 2026).

Bokeh Development Team, 2024. *Bokeh: Python library for interactive visualization.* Available at: https://bokeh.org (Accessed: 18 March 2026).

Chacon, S. and Straub, B., 2014. *Pro Git*. 2nd ed. Berkeley, CA: Apress.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J., 1994. *Design Patterns: Elements of Reusable Object-Oriented Software.* Boston: Addison-Wesley.

Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J. and others, 2020. Array programming with NumPy. *Nature,* 585(7825), pp.357–362.

McKinney, W., 2022. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter.* 3rd ed. Sebastopol: O'Reilly Media.

Montgomery, D.C., Peck, E.A. and Vining, G.G., 2012. *Introduction to Linear Regression Analysis.* 5th ed. Hoboken: Wiley.

Myers, G.J., Sandler, C. and Badgett, T., 2011. *The Art of Software Testing*. 3rd ed. Hoboken: Wiley.

Owens, M., Allen, G. and Kennedy, C., 2010. *The Definitive Guide to SQLite.* Berkeley: Apress.

Van Rossum, G. and Drake, F.L., 2009. *Python 3 Reference Manual.* Scotts Valley: CreateSpace.